

Requirements

Technical stack

- **Java 17**
- **Spring Boot** (latest stable)
- **Maven or Gradle** (your choice)
- **H2 in-memory database** (no external database setup required)

Your API must include

1. Task CRUD endpoints

A `Task` has at minimum:

- `id` (auto-generated)
- `title` (string, required)
- `description` (string, optional)
- `dueDate` (date)
- `priority` (LOW / MEDIUM / HIGH)
- `status` (TODO / IN_PROGRESS / DONE)

Implement the following endpoints:

Method	Path	Description
-----	-----	-----
<code>POST</code>	<code>/tasks</code>	Create a new task
<code>GET</code>	<code>/tasks</code>	List all tasks
<code>GET</code>	<code>/tasks/{id}</code>	Get a single task
<code>PUT</code>	<code>/tasks/{id}</code>	Update a task
<code>DELETE</code>	<code>/tasks/{id}</code>	Delete a task

2. At least one AI-powered endpoint

Integrate a call to an AI model (Claude, GPT-4, Gemini, or similar) within your API. You may choose your approach — here are some ideas to get you started, but you are welcome to come up with your own:

- `POST /tasks/suggest` — accepts a plain-language description (e.g., `"remind me to submit the quarterly report before Friday"`) and returns a structured task object
- `POST /tasks/{id}/summarize` — returns a plain-language summary or explanation of a task
- `POST /tasks/{id}/breakdown` — returns a suggested list of subtasks for a complex task

Your AI-powered endpoint should return well-structured JSON. It does not need to persist anything — a stateless call to the model is fine.

****3. A simple UI****

Include a minimal frontend that allows a reviewer to interact with your API without using a REST client. It does not need to be polished or feature-complete — a basic HTML page is fine. It should support:

- Viewing the list of tasks
- Creating a new task
- Triggering your AI-powered endpoint and displaying the result

Styling and aesthetics are not evaluated. The UI exists purely to make your project easier to explore.

****4. Basic tests****

Include a test suite that covers the core behavior of your API. At minimum:

- At least one unit test per service-layer method (happy path)
- At least one integration test that starts the Spring context and exercises each CRUD endpoint end-to-end
- At least one test for your AI-powered endpoint (mocking the external model call is fine and expected)

Tests must pass when running: